



OBJECTIVE

Build a high-concurrency, multi-user voice AI gateway. The core pipeline must be built around the [Pipecat Quickstart Architecture](#).

- The system must achieve $\leq 2s$ E2E latency while implementing distributed state, JWT auth, and Redis-backed rate limiting.

PART 1 – DISTRIBUTED BACKEND

Build a Python service that:

- Session Management: Implement a `SessionManager` to map WebSockets to `PipelineTask` instances.
- Concurrency: Support multiple simultaneous users/streams without blocking the FastAPI event loop.
- Tool Execution: Implement a `Play Audio` tool within the LLM context that triggers immediate binary frames to the client.
- Usage Tracking: Calculate real-time STT seconds, LLM tokens, and TTS characters per session.

PART 2 – SECURITY & TRAFFIC CONTROL

- JWT Auth: Implement `/token` issuance and mandatory Bearer validation for the `/ws/talk` upgrade.
- Distributed Rate Limiting: Use Redis to enforce a "Concurrency Cap" (e.g., 2 active calls/user) and usage quotas.
 - Return WebSocket close code `4001` (Policy Violation) on limit breach.

PART 3 – DEPLOYMENT

- Provide a Dockerfile with all dependencies
- App must run with one command after cloning
- No platform-specific dependencies
- Must run on Windows, macOS, Linux (make sure to use)

PART 4 – OBSERVABILITY & COST VISIBILITY

Include:

- Structured logging
- Clean session teardown on disconnect
- Logged and exposed per-turn and per-conversation cost estimation



PART 5 – DOCUMENTATION

`README.md` must detail:

- Scaling: Strategy for 5,000+ concurrent calls (Gateway vs. Worker node separation).
- Resilience: Circuit-breaker logic for upstream AI provider failures.
- Optimization: Techniques used to minimize hot-path overhead during JWT/Redis checks.

DEMO VIDEO (REQUIRED)

Provide a 3–5 minute technical demo video showing:

- Authentication Flow: Demonstrate JWT rejection (unauthorized) and successful connection with a valid token.
- Multi-User Concurrency: Two separate browser tabs or clients interacting with the agent simultaneously.
- Traffic Control: Triggering the Redis rate-limiter by exceeding the concurrency cap or usage quota.
- Observability: A walkthrough of the `/metrics` endpoint showing real-time latency histograms and cost tracking.

Requirements:

- Screen recording with audio explanation.
- Single take preferred; no high-production editing required.
- Link included in the `README.md`.

DELIVERABLES

- Source: FastAPI code (Auth, Session, and Pipeline modules).
- Infra: `Dockerfile` and `docker-compose.yml`.
- Docs: Technical rationale and scaling roadmap.
- Demo: Video showing multi-user concurrency, JWT rejection, rate-limiting, and live metrics.

EVALUATION CRITERIA

- End-to-end latency $\leq 2s$ (primary)
- Streaming & async design quality
- Correctness under concurrency
- Tool-calling implementation quality
- Cost awareness and estimation clarity
- Production readiness & observability
- Code quality and maintainability